

FL-MPS: Secure Federated LLM Fine-Tuning via Adaptive Parameter Split and Efficient Negotiation

Yuejuan Zhu , Yunlong Mao , *Member, IEEE*, Guanzhi Qiu, Shijie Ding, Wenjun Qian ,
and Sheng Zhong , *Fellow, IEEE*

Abstract—Large Language Models (LLMs) are widely deployed across various domains and multi-agent systems, where personalized fine-tuning is often required to adapt models to user-specific data. However, centralized LLM fine-tuning is constrained by limited access to diverse data and the high computational cost of large-scale training. Federated learning (FL), combined with parameter-efficient tuning methods such as LoRA, enables distributed model adaptation while keeping data local. Nevertheless, uploading LoRA updates in plaintext exposes sensitive information, while fully homomorphic encryption introduces substantial computation and communication overhead. To address this challenge, we propose FL-MPS, a secure and communication-efficient aggregation framework for federated LoRA fine-tuning. FL-MPS consists of two stages: a parameter-level negotiation stage, where clients exchange hashed anchors and importance indicators to determine the protection mode for each LoRA component, and a secure aggregation stage based on our proposed Residual-Based PHE, which encrypts only the necessary parameter segments for aggregation. Experimental results show that FL-MPS reduces homomorphic encryption overhead compared with full parameter encryption while achieving comparable or improved performance over state-of-the-art baselines.

Index Terms—Federated learning, large language models, security, homomorphic encryption, low-rank adaptation.

I. INTRODUCTION

LARGE Language Models (LLMs) such as ChatGPT [1] have demonstrated remarkable capabilities across a wide range of tasks, including natural language understanding [2] and multimodal generation [3]. While these models exhibit strong generalization ability, deploying them in real-world applications often requires adaptation to domain-specific data and user-specific contexts through fine-tuning. This need arises in many practical scenarios, such as legal assistance [4], medical diagnosis [5], and financial analysis, where models must handle

specialized terminology and sensitive data. However, the data required for such fine-tuning is typically distributed across multiple institutions or user devices, making centralized training difficult.

However, most high-quality datasets are stored in decentralized institutions such as hospitals, banks, and mobile devices, where strict privacy regulations (e.g., GDPR, CCPA) [6], [7] prevent the sharing of raw data with centralized servers. This creates a tension between the need for large, high-quality datasets and the requirement for privacy, driving the adoption of Federated Learning (FL) [8] as a promising solution. FL enables the collaborative training of models without requiring data to leave its source, thus preserving privacy and complying with privacy laws.

Despite its advantages in privacy preservation, federated fine-tuning of LLMs still faces substantial bottlenecks in terms of resource consumption. Full-parameter fine-tuning [9] requires vast amounts of memory and communication bandwidth, making it impractical for edge devices or large-scale federated deployments. Moreover, uploading the full parameters during updates introduces privacy risks. This vulnerability stems from the potential of LLMs to unintentionally disclose sensitive information through their generated outputs [10].

To mitigate these issues, recent approaches have leveraged Low-Rank Adaptation (LoRA) [11] in conjunction with FL. LoRA allows clients to update only a set of lightweight low-rank matrices while keeping the backbone model frozen, which significantly reduces the communication and computation overhead, making it an ideal solution for federated LLM fine-tuning. The introduction of LoRA into FL [12], [13] makes large-scale federated training more feasible. However, model updates are still susceptible to inference and gradient inversion attacks (GIA) [14], which may result in the disclosure of private information.

To ensure end-to-end privacy in federated LLM training, some approaches adopt Differential Privacy (DP) [15]. However, DP introduces noise that can severely degrade the model's performance, especially in LLM fine-tuning, where accuracy and precision are critical [15]. Another solution is Fully Homomorphic Encryption (FHE) [16], which enables encrypted computations on model updates, providing strong privacy guarantees. FedShield-LLM [16] improves privacy in federated LoRA fine-tuning by enforcing full homomorphic encryption and pruning. However, it treats LoRA parameters homogeneously, leading to unnecessary cryptographic overhead.

Received 11 March 2026; revised 28 June 2026; accepted 29 July 2026. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62272222 and Grant 62272215, in part by the Jiangsu Province Outstanding Youth Fund Project under Grant BK20230080, and in part the Nanjing UniversityChina Mobile Communications Group Company, Ltd. Joint Institute under Grant NJ20250025. Recommended for acceptance by Yuezhi Che. (Corresponding authors: Yunlong Mao; Wenjun Qian.)

Yuejuan Zhu, Yunlong Mao, Guanzhi Qiu, Shijie Ding, and Sheng Zhong are with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: ylmiao@nju.edu.cn).

Wenjun Qian is with China Mobile Research Institute, Beijing 100053, China (e-mail: qianwenjun@chinamobile.com).

Digital Object Identifier 10.1109/TDSC.2026.3719352

In addition to DP- and HE-based solutions, FLSwitch [17] represents a recent attempt to improve the efficiency of secure aggregation by dynamically switching protection strategies. It adopts a two-stage design consisting of a negotiation phase and an aggregation phase, and leverages Residual-Based Partially Homomorphic Encryption (RBPHE) to encrypt model updates. A learned decision model is employed to enforce discrete switching between encryption states during training. While effective in small- and medium-scale settings, such model-driven state transitions may introduce abrupt changes in optimization dynamics, leading to training instability. Moreover, FLSwitch is primarily designed for relatively lightweight models and is difficult to scale to large language model fine-tuning, where parameter sensitivity, numerical precision, and communication costs exhibit fundamentally different characteristics.

These limitations motivate us to revisit a fundamental intuition behind HE-based secure aggregation: under strict security guarantees, improving efficiency in federated large language model fine-tuning essentially boils down to reducing the amount of data involved in ciphertext-domain operations. Prior work has explored sparse or compressed parameter representations to alleviate this overhead [16]. However, such strategies are ill-suited for the FedLoRA setting, as LoRA parameters are already low-rank and compact by design. Further sparsification would severely degrade model accuracy, raising a natural question: Is it fundamentally impossible to reduce secure aggregation overhead in FedLoRA?

To answer this question, we propose FL-MPS, a new and more flexible secure aggregation framework tailored for federated LoRA fine-tuning. Our design is built upon two key observations. First, during fine-tuning, LoRA parameters uploaded by different clients exhibit heterogeneous behaviors in terms of numerical range and precision: while their value ranges remain relatively stable across clients, their high-precision fractional components vary significantly. Second, in gradient inversion and model extraction attacks [18], adversaries benefit more from recovering coarse-grained range information than from accurately reconstructing high-precision residuals.

Motivated by these observations, FL-MPS introduces a parameter-splitting encryption paradigm, which enables adaptive and negotiation-based protection for LoRA parameters. Specifically, clients collaboratively determine the encryption strategy at the parameter level. This adaptive design significantly reduces ciphertext-domain computation and communication costs, while preserving robust security guarantees and model utility.

The Fig. 1 shows an overview of the FL-MPS framework. It includes the two primary phases: negotiation and aggregation, where the negotiation phase focuses on deciding which parameters should be encrypted fully, which should have anchors encrypted, and which should only transmit residuals. In the aggregation phase, RBPHE is used to securely aggregate the encrypted data, ensuring both privacy and efficiency.

To achieve the above objective, the key challenge lies in determining an appropriate parameter splitting point, i.e., deciding which bits of each parameter should be encrypted. Encrypting fewer bits leads to greater efficiency gains, as it reduces

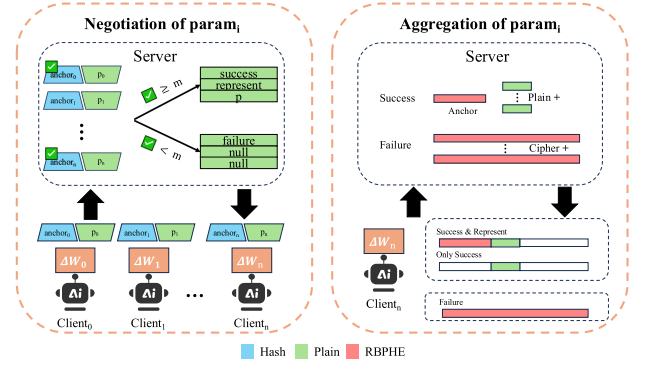


Fig. 1. Workflow illustration of FL-MPS.

ciphertext-domain computation and communication costs; however, it also increases potential privacy risks. Conversely, encrypting more bits enhances security but diminishes the efficiency benefits of partial encryption. Therefore, an essential goal of FL-MPS is to identify a balanced parameter-splitting strategy that jointly optimizes security and efficiency.

Another important observation is that, as federated training progresses and the global model gradually converges, different clients may produce identical or highly similar parameter components that require encryption. This phenomenon becomes increasingly prevalent in later training rounds, especially for stable parameter components. Exploiting this redundancy opens up further opportunities for reducing the amortized cost of secure aggregation. Consequently, during the negotiation phase, clients are required not only to locally determine a minimum number of bits that must be encrypted to meet their security requirements, but also to coordinate with the server to identify whether identical encrypted components exist across clients. If such an agreement is reached, a representative encrypted upload can be selected during aggregation, thereby further reducing redundant ciphertext transmission and computation.

Building upon the negotiation stage, we use Residual-Based Partially Homomorphic Encryption (RBPHE) to encrypt the parameters selected by local clients, and apply homomorphic addition to aggregate the encrypted data. The residual portion is uploaded, whose length is based on the negotiated precision, further reducing computation and communication costs. We summarize our contributions as follows:

- *Privacy-preserving PEFT Scheme:* We design a privacy-preserving parameter-efficient fine-tuning (PEFT) scheme for federated LLM fine-tuning, offering provable security guarantees.
- *Efficiency Optimization:* To optimize efficiency, we propose a dynamic parameter partitioning strategy and a highly efficient negotiation algorithm. This approach significantly enhances performance without compromising security.
- *Extensive Empirical Validation:* We conduct extensive experiments on LLM fine-tuning tasks, specifically using Shepherd-7B with the Databricks-dolly-15 k and Wizard of Wikipedia dataset. Our experiments demonstrate a slight performance improvement over multiple baseline methods,

striking a favorable balance between privacy protection, communication efficiency, and model performance.

II. PRELIMINARY AND MOTIVATION

A. Federated Learning With LoRA

In the federated fine-tuning of large language models (LLMs), the overall training procedure follows the standard FL workflow, while restricting local optimization to a small set of low-rank LoRA parameters. At the beginning of the t -th training iteration, each participant $P_i, i \in [1, N]$ downloads the latest globally aggregated LoRA parameters $\bar{\theta}_t$ from the parameter server (PS). Since the backbone of the LLM is kept frozen during fine-tuning, the only trainable components on each client are the LoRA modules. We denote these trainable parameters collectively as $\Delta\theta_t^i$, which represent the low-dimensional adaptation injected into selected weight matrices of the LLM.

Given the received global model, participant P_i performs local training using its private dataset D_i and constructs a mini-batch $\{x_1, x_2, \dots, x_B\}$ of size B . The local objective is to minimize the loss $\mathcal{L}_i(\bar{\theta}_t + \Delta\theta_t^i)$, where the optimization is carried out only over LoRA parameters. Formally, the gradient of the LoRA components is computed as

$$\mathbf{g}_t^i = \frac{1}{B} \sum_{j=1}^B \nabla_{\Delta\theta_t^i} \mathcal{L}_i(\bar{\theta}_t + \Delta\theta_t^i, x_j),$$

and the LoRA update is performed via

$$\Delta\theta_{t+1}^i = \Delta\theta_t^i - \eta \mathbf{g}_t^i.$$

In LoRA, each adapted weight matrix $\mathbf{W}$ is equipped with a pair of low-rank factors $(\mathbf{A}_i, \mathbf{B}_i)$ such that the effective update can be written as

$$\Delta\mathbf{W}_i = \mathbf{A}_i \mathbf{B}_i,$$

where the rank r satisfies $r \ll \min(d, k)$. After local training, participant P_i uploads only the LoRA-related updates $\Delta\theta_{t+1}^i$ to the PS. Assuming all participants contribute equally, the PS aggregates the received LoRA updates to obtain the next global model:

$$\bar{\theta}_{t+1} = \bar{\theta}_t + \frac{1}{N} \sum_{i=1}^N \Delta\theta_{t+1}^i.$$

Since $\Delta\theta_{t+1}^i$ is substantially smaller than the full LLM parameter space, LoRA significantly reduces the communication and computation costs of FL training. However, despite its compact size, the LoRA update vector still contains rich gradient information, which may reveal sensitive patterns from D_i under model inversion or reconstruction attacks. This motivates the need for secure aggregation mechanisms specifically designed for LoRA-based federated LLM fine-tuning.

B. Threat Model

Following previous studies [19], [20], [21], we design FLSMA-LoRA in a semi-honest setting, assuming that both the

PS and participants could be honest but curious. A secure communication channel is assumed to be available between the PS and each participant. A random secret seed can be pre-installed by a certificate authority or by running a distributed protocol between participants only once for secret key generation during the training. If the PS is adversarial, then no collusion with any adversarial participant is allowed in the semi-honest setting. However, we allow the collusion of adversarial participants of up to $N - 2$ when PS is semi-honest. We will focus on the model confidentiality of participants in the discussion about adversaries, just like in previous studies [19], [20]. The correctness and verification of learning will not be discussed here and should be studied separately [22], [23].

Adversarial goal: The adversary is to disclose the private information of a target participant. There are plenty of potential attacks against FL participants, such as membership inference [24], [25], property inference [26], [27], and data reconstruction [27], [28] attacks. Some of these attacks should be handled by a mixture of HE and DP techniques. However, DP based solution is orthogonal to our study and should be discussed separately. Hence, we simplify the adversarial goal as disclosing a target participant's local model updates, precisely, model parameters θ or the corresponding gradients $\mathbf{g}$.

C. Homomorphic Encryption

We note that our HE-based SMA is implemented on the basis of the original Paillier HE (PHE). But it can also be adapted to other HE systems. For simplicity, we initialize a general PHE system as follows.

- $PSetup(\lambda) \rightarrow (pk, sk)$: Given a security parameter λ , the algorithm generates a pair of public and secret keys (pk, sk) .
- $PEncrypt(pk, v) \rightarrow c$: Taking as input a value v and a public key pk , the algorithm outputs a ciphertext c .
- $PDecrypt(sk, c) \rightarrow m$: Taking as input a ciphertext c and a secret key sk , the algorithm outputs the decrypted value v .
- $PAdd(c_1, c_2) \rightarrow c'$: Taking as input two ciphertexts c_1, c_2 , the algorithm outputs a ciphertext satisfying $PDecrypt(c_1, sk) + PDecrypt(c_2, sk) = PDecrypt(c', sk)$.

D. Observations

Observation 1: LoRA A exhibits high cross-client consistency, while LoRA B captures client-specific heterogeneity in federated learning:

Prior studies have observed that different components of LoRA play distinct functional roles in adaptation. In particular, existing work shows that, under the federated learning setting, the LoRA A matrices learned by different clients tend to be highly similar even when clients are trained on heterogeneous datasets, whereas the LoRA B matrices exhibit substantially larger divergence across clients [29].

This observation suggests that treating LoRA parameters uniformly during aggregation is unnecessary and potentially inefficient. Instead, parameters should be handled differently,

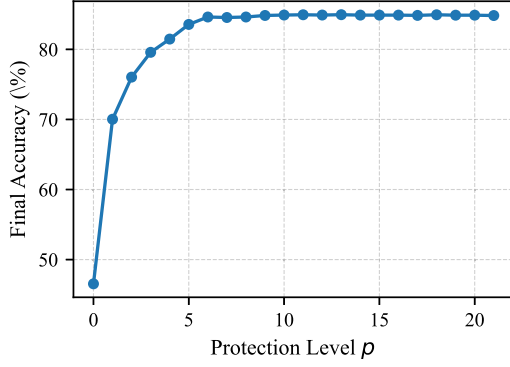


Fig. 2. Impact of p on the final accuracy.

with aggregation strategies explicitly accounting for their distinct statistical and semantic properties across heterogeneous clients.

Observation 2: Most of the effective information in LoRA parameters is preserved by the high-order mantissa bits:

Beyond structural heterogeneity across LoRA components, we further investigate redundancy at the numerical representation level. During the aggregation stage, we analyze uploaded LoRA parameters represented in floating-point format by progressively masking the least significant bits of the mantissa.

Fig. 2 shows that retaining only the first 5 bits of the mantissa yields aggregated models whose performance remains highly consistent with that obtained using full-precision parameters. p corresponds to the number of most significant mantissa bits that remain unmasked in the uploaded LoRA parameters. Further preserving additional lower-order bits brings negligible performance gains, indicating that these bits contribute little to the effective model update.

This observation implies that LoRA updates exhibit substantial numerical redundancy, where low-significance mantissa bits have minimal impact on aggregation outcomes. Such redundancy opens up opportunities for more efficient parameter transmission and protection mechanisms, as aggressive truncation of low-order bits can significantly reduce communication or encryption overhead without compromising model quality.

III. METHODOLOGY

A. Overview

We propose FL-MPS, a secure and communication-efficient aggregation framework specifically designed for federated LoRA fine-tuning. In FL-MPS, we leverage the inherent stability differences across LoRA parameters in order to apply varying levels of protection to different components. Some LoRA parameters converge quickly and remain stable throughout training, while others fluctuate significantly due to the non-IID nature of the data. Our framework adapts to these variations by selecting the appropriate aggregation and encryption strategies for each parameter based on its stability.

The FL-MPS framework consists of two main stages: a negotiation-driven secure aggregation stage and a secure aggregation stage. During the negotiation stage, clients collaboratively

determine the protection mode for each LoRA parameter. The possible protection strategies include:

- Fully homomorphic encryption.
- Encryption by a representative client only.
- Uploading as a low-bit residual.

At each round, clients compute lightweight hashed anchors from the significant bits of each LoRA parameter and generate a precision indicator according to local importance scores based on gradient-weight interactions [30]. These summaries are sent to PS for negotiation. If a sufficiently large number of clients share the same anchor, the parameter is considered to have reached a consensus. In this case, only one representative client performs homomorphic encryption on the high-order bits, while all clients upload only the low-bit residuals. The precision of these residuals is determined dynamically based on the negotiated indicator, reducing the need for communication.

For parameters that fail to reach consensus—typically unstable or divergent LoRA components—fully homomorphic encryption is enforced to protect sensitive updates from exposure. This selective encryption strategy ensures that only the necessary components are fully encrypted, avoiding unnecessary encryption of the entire LoRA matrix for each client. The approach results in significant reductions in both computational and communication overhead, making federated LoRA fine-tuning more efficient and scalable.

B. Homomorphic Encryption

1) *Residue-Based PHE Scheme (RBPHE)*: We now introduce an efficient residue-based PHE scheme RBPHE. The goal of RBPHE is to pack a batch of fixed-point numbers into one ciphertext within a flexible encoding range, supporting SIMD operations. We consider the overflow and quantization of gradients in real training scenarios, where the gradients follow the nearly Gaussian distribution [19], [31]. BatchCrypt [19] reserves enough bits according to the number of parties to avoid overflow. Thus, the solution is limited to scenarios where the addition number should be predefined. FLASHE [32] uses a STATEful symmetric scheme and assumes a threat model where the aggregation server does not collude with any party. As for FHE-based solutions, such as CKKS [33], can reach the lowest encryption overhead. However, the ciphertext size is much larger than others, which results in large memory and communication overhead.

Compared with the existing schemes, RBPHE advances in two aspects. On one side, RBPHE takes less amortized overhead for critical operations. On the other side, RBPHE provides an automatic and flexible encoding range extension method, balancing parameter precision and efficiency. In this way, RBPHE achieves a more efficient batching than the existing schemes. Moreover, since RBPHE avoids gradient clipping through dynamic range extension, more precise model updates will be preserved.

Generally, RBPHE utilizes a residue number system (RNS) to encode a batch of parameters. Specifically, real numbers are converted to residues with a predefined prime modulus so that multiple residues can be encoded into a large integer and referred

to as a package through the Chinese Remainder Theorem (CRT). We use two moduli (with a particular condition) for each real number and an extra pair of prime moduli to count the addition operations within batching. Therefore, RBPHE will convert each batch of real numbers into two packages. Instead of directly encrypting two packages using PHE, a random mask is used to randomize one package and then encode it into the other package. In this way, we only encrypt the unmasked package to reduce the number of homomorphic operations and improve efficiency.

Intuitively, homomorphic addition will lead to an overflow when the aggregated residue of two encoded real numbers is larger than the prime modulus. To correctly decode addition results beyond the encoding range, we leverage the observation that a small decoding difference will be generated when the sum of two residues is larger than their modulus. Since each real number is encoded using two moduli, a unit difference between a pair of moduli will be detected whenever the encoded residue grows larger than its modulus. Therefore, we can eliminate the effect of overflow by counting unit differences and recovering the original real number. We now present a practical implementation of RBPHE.

Setup($\lambda, \mathcal{L}, \mathcal{T}, \mathcal{B}$): The algorithm takes as input a security parameter λ , an encoding bit length $\mathcal{L}$, a maximum addition time $\mathcal{T}$, and a batching size $\mathcal{B}$, and outputs public parameters $\{\mathcal{P}, \mathcal{Q}\}$ and (pk, sk) .

- 1) Set $\mathcal{T}' = \mathcal{T} \cdot 2^\lambda$. Pick two primes $p_0 > \mathcal{T}'$, $q_0 > \mathcal{T}$ and two set of primes $\{p_1, p_2, \dots, p_B\}$, $\{q_1, q_2, \dots, q_B\}$ satisfying $p_i > 2^\mathcal{L}$ and $(q_i - 1) = k_i(p_i - 1)$, where integer $k_i > 1$.
- 2) Run $PSetup(\lambda)$ to obtain (pk, sk) .
- 3) Set $\mathcal{P} = \{p_i | 0 \leq i \leq B\}$, $\mathcal{Q} = \{q_i | 0 \leq i \leq B\}$ and output $(pk, sk, \{\mathcal{P}, \mathcal{Q}\})$.

Encrypt($R, \{\mathcal{P}, \mathcal{Q}\}, pk, \theta$): The algorithm takes as input an encoding range R , parameters θ , and $\{\mathcal{P}, \mathcal{Q}\}$, pk , and outputs a ciphertext c , if $|\theta| \leq B$. Otherwise, it outputs $\perp$.

- If $|\theta| > |\mathcal{P}|$, directly output $\perp$. Otherwise, pick a mask $r \xleftarrow{\$} \{0, 1\}^\lambda$ uniformly at random and convert each $\theta_i \in \theta$ to residues with $p_i \in \mathcal{P}$, $q_i \in \mathcal{Q}$ by the following two equations (where $\theta_i = 0$ for $i > |\theta|$):

$$\langle \theta_i \rangle_{p_i} = \left\lfloor \frac{\theta_i + R}{2R} \cdot (p_i - 1) \right\rfloor + r \cdot \frac{p_i - 1}{2} \pmod{p_i},$$

$$\langle \theta_i \rangle_{q_i} = \left\lfloor \frac{\theta_i + R}{2R} \cdot (q_i - 1) \right\rfloor + r \cdot \frac{q_i - 1}{2} \pmod{q_i}.$$

- Set $\langle \theta_0 \rangle_{p_0} = r$, $\langle \theta_0 \rangle_{q_0} = 1$, $\mathcal{R}_1 = \{\langle \theta_i \rangle_{p_i} | 0 \leq i \leq B\}$, $\mathcal{R}_2 = \{\langle \theta_i \rangle_{q_i} | 0 \leq i \leq B\}$ and evaluate $\mu_1 \leftarrow crt(\mathcal{P}, \mathcal{R}_1)$, $\mu_2 \leftarrow crt(\mathcal{Q}, \mathcal{R}_2)$, where crt is the abstracted function of CRT.
- Run $c_1 \leftarrow PEncrypt(pk, \mu_1)$ and output $c = (c_1, \mu_2)$.

Decrypt($\{\mathcal{P}, \mathcal{Q}\}, sk, c$): The algorithm takes as input a ciphertext c and $\{\mathcal{P}, \mathcal{Q}\}$, sk , and outputs parameters θ , if sk and c are valid. Otherwise, it outputs $\perp$.

- Parse $c = (c_1, \mu_2)$. If $PDecrypt(sk, c_1)$ outputs $\perp$, the algorithm directly outputs $\perp$. Otherwise, it obtains $\mu_1 \leftarrow PDecrypt(sk, c_1)$.

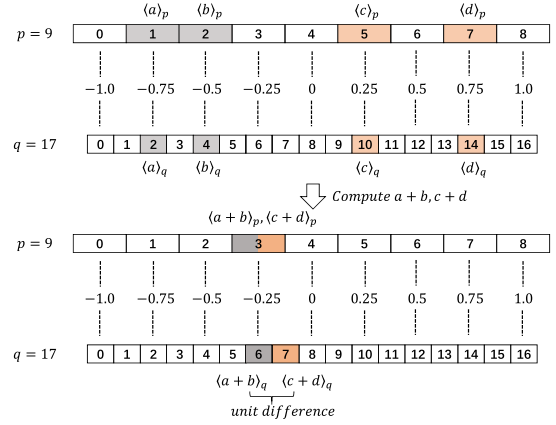


Fig. 3. An example of unit difference generated by homomorphic addition, $p = 9, q = 17$ (q is not required to be a prime in this case).

- For $i \leftarrow 1$ to N :
 - 1) Compute $\langle \theta_i \rangle_{p_i} = \mu_1 - r \cdot \frac{p_i - 1}{2} \pmod{p_i}$, $\langle \theta_i \rangle_{q_i} = \mu_2 - r \cdot \frac{q_i - 1}{2} \pmod{q_i}$, $k_i = \frac{q_i - 1}{p_i - 1}$.
 - 2) If $\langle \theta_i \rangle_{q_i} < \langle \theta_i \rangle_{p_i} \cdot k_i$, set $\langle \theta_i \rangle_{q_i} = \langle \theta_i \rangle_{q_i} + q_i$.
 - 3) Compute unit difference $unit_i = k_i - 1$ and overflow time $t_i = \frac{\langle \theta_i \rangle_{q_i} - k_i \cdot \langle \theta_i \rangle_{p_i}}{k_i - 1}$.
 - 4) Recover the value $\theta_i = (\langle \theta_i \rangle_{p_i} + t \cdot p - M \cdot \frac{p-1}{2}) \cdot \frac{2}{p-1}$.
- Output $\{\theta_i | 1 \leq i \leq B\}$.

Add(c, c'): The algorithm takes as input two ciphertexts c and c' and outputs a new ciphertext c_{Add} , satisfying $Decrypt(sk, c_{Add}) = Decrypt(sk, c) + Decrypt(sk, c')$.

- Parse $c = (c_1, \mu_2)$, $c' = (c'_1, \mu'_2)$.
- Evaluate $c_{Add} \leftarrow PAdd(c_1, c'_1)$ and $\mu_{Add} = \mu_2 + \mu'_2$.
- Output (c_{Add}, μ_{Add}) .

Mul(c, β) $\rightarrow c'$: The algorithm takes a ciphertext c and an integer coefficient $\beta \in \mathbb{Z}$ as inputs, it outputs a new ciphertext c' so that $Decrypt(sk, c') = Decrypt(sk, c) \otimes \{\beta, \beta, \dots\}$, where $\otimes$ indicates the element-wise multiplication.

The proposed RBPHE inherits the homomorphic addition algorithm **Add** from PHE by leveraging the homomorphism of RNS. In particular, a basic prime pair $\langle p_0, q_0 \rangle$ is used as a counter of addition operations for the encoded elements. All encoded elements and their addition times will be added when performing element-wise addition on two ciphertexts for the correctness of decoding. To correctly decode the addition results beyond the encoding range $[-R, R]$, resolving the overflow issue, we leverage the observation that a small decoding difference will be generated when the sum of two residues is larger than their modulus. Since each real number is encoded with two moduli, a unit difference between a pair of moduli p_i, q_i will be detected whenever the encoded residue grows larger than its modulus. Therefore, we can eliminate the effect of overflow by counting the number of unit differences and recovering the original real number. (an example is shown in Fig. 3).

Generally, RBPHE utilizes a residue number system (RNS) to encode a batch of parameters into a large integer. Specifically, in the **Encrypt** algorithm, real numbers $\{g_i\}$ are converted to

residues with predefined prime modulus so that multiple residues can be encoded into a large integer and referred to as a package through the Chinese Remainder Theorem (CRT). We set two modulus p_i, q_i (with a special condition $(q_i - 1) = k_i(p_i - 1)$) for each real number and use an extra pair of prime modulus p_0, q_0 to count the addition operations when packing a batch of real numbers. Therefore, each batch of real numbers will be converted into two packages μ_1, μ_2 by RBPHE. Instead of directly encrypting two packages using PHE, a random mask r will be generated to randomize one package and encode it in the other package. In this way, we can only encrypt the unmasked package to reduce the number of homomorphic operations and improve efficiency. Please note that RBPHE naturally supports homomorphic addition due to the homomorphism of RNS and PHE.

The proposed RBPHE inherits the homomorphic addition algorithm *Add* from PHE by leveraging the homomorphism of RNS. In particular, a basic prime pair $\langle p_0, q_0 \rangle$ is used as a counter of addition operations for the encoded elements. All encoded elements and their addition times will be added when performing element-wise addition on two ciphertexts for the correctness of decoding. Intuitively, homomorphic addition will lead to an overflow when the aggregated residue of two encoded real numbers is larger than the prime modulus. To correctly decode the addition results beyond the encoding range $[-R, R]$, we leverage the observation that a small decoding difference will be generated when the sum of two residues is larger than their modulus. Since each real number is encoded with two moduli, a unit difference (shown in step 3) between a pair of moduli p_i, q_i will be detected whenever the encoded residue grows larger than its modulus. Therefore, we can eliminate the effect of overflow by counting the amount of unit differences and recovering the original real number.

As for the multiplicative homomorphism of RBPHE, the implementation only supports the multiplication of a ciphertext and a constant integer vector. Besides, the addition times for all elements in the encoded number should be equal. Fortunately, RBPHE can be extended to support element-wise integer multiplication by introducing multiple basic prime pairs, as (p_0, q_0) picked in the *Setup* algorithm, to record addition times for each element. However, the additional prime pairs will decrease the maximum batch size and affect efficiency, which is not necessary for our study.

RBPHE supports batching a vector of gradients into a single ciphertext and performing element-wise homomorphic operations in the ciphertext. In RBPHE, amortized overhead is affected by the encoding batch size $\mathcal{B}$, namely the number of gradients in each ciphertext. Although a larger batch size will improve the efficiency of the exploring phase, the upper bound of the batch size is determined by the security and precision of RBPHE, which will be discussed in Section IV-A. Security of RBPHE.

2) *Encoding Precision and Batching Size*: Since each parameter θ_i is encoded using two primes p_i, q_i , satisfying $2^{\mathcal{L}} < p_i < q_i$, the encoding precision is determined by the smaller prime p_i . Assuming that the encoding range is $[-R, R]$, the precision can be implicitly inferred by $\frac{2R}{p_i-1} \leq \frac{R}{2^{\mathcal{L}-1}}$. If the rounding operation $\lceil \cdot \rceil$ is used to round an input to its nearest integer, then the upper

Algorithm 1: Parameter-Level Negotiation of FL-MPS.

INPUT: Number of clients N , threshold m , bits of sign and exponent S

OUTPUT: Negotiation table $T = \{(s_j, r_j, p_j)\}_{j=1}^M$

ClientProposal i

- 1: Perform local LoRA fine-tuning and obtain (A_i, B_i)
- 2: Flatten parameters: $\theta_i \leftarrow \text{flatten}(A_i, B_i)$
- 3: **for** $j \leftarrow 1$ **TO** M **do**
- 4: Compute the importance score and derive precision p_j^i
- 5: Extract anchor bits: $a_j^i \leftarrow \text{ExtractBits}(\theta_j^i, 0, S)$
- 6: Compute the hash: $h_j^i \leftarrow \text{Hash}(a_j^i)$
- 7: **end for**

- 8: Upload $\{(h_j^i, p_j^i)\}_{j=1}^M$ to the server

ServerNegotiation

- 9: **for** $j \leftarrow 1$ **TO** M **do**
 - 10: Collect hashes $\{h_j^i\}_{i=1}^N$
 - 11: **if** at least m clients share an identical hash **then**
 - 12: $s_j \leftarrow 1$
 - 13: $r_j \leftarrow$ selected client with the majority hash
 - 14: $p_j \leftarrow \text{negotiate}(\{p_j^i\}_{i=1}^N)$
 - 15: **else**
 - 16: $s_j \leftarrow 0, r_j \leftarrow -1, p_j \leftarrow \emptyset$
 - 17: **end if**
 - 18: **end for**
 - 19: Broadcast negotiation table T to all clients
-

bound of the maximum decoding error should be $\frac{R}{p_i-1} \leq \frac{R}{2^{\mathcal{L}}}$. However, RBPHE can flexibly extend its encoding range when necessary. Intuitively, leveraging the addition times encoded with q_0 , the maximum encoding range can be extended to $[-q_0 R, q_0 R]$. In this way, the maximum addition time regarding q_0, p_i, q_i can be computed by $\min(q_0, \lfloor \frac{q_i(p_i-1)}{q_i-p_i} \rfloor)$, which is usually large enough for real-world FL applications.

The maximal batching size is determined by the key size of PHE, the maximum addition time $\mathcal{T}$, and the primes used in RBPHE. Assuming $\mathbb{Z}_{\mathcal{K}}$ is the input space of PHE according to the security parameter λ , $\mu_1 \leq \mathcal{K}$ should hold to guarantee the correctness of RBPHE. We can first generate enough, say as $\mathcal{B}'$, primes $\tilde{\mathcal{P}} = \{p_i | 1 \leq i \leq \mathcal{B}'\}$ under a given bit length $\mathcal{L}$. Note that each prime p_i satisfies the condition that there exist another prime q_i and an integer k_i holding $q_i - 1 = k_i(p_i - 1)$. Besides, p_0 can also be determined by $\mathcal{T} \cdot 2^\lambda$. Therefore, the maximal batching size can be determined by increasing $\mathcal{B}$ until $\prod_{i=0}^{\mathcal{B}} p_i \geq \mathcal{K}$. The minimal batching size is related to the security of RBPHE and will be discussed in Section IV-A. Security of RBPHE.

C. Negotiation Phase

The negotiation phase in FL-MPS is a collaborative process where clients exchange key information to dynamically determine the appropriate protection mode for each LoRA parameter. This phase is crucial for ensuring both privacy and computational efficiency, as it enables clients to adaptively select encryption strategies based on parameter stability, reducing unnecessary

overhead during the aggregation phase. During each training round, the negotiation phase proceeds through two stages, anchor negotiation and precision indicator negotiation, which are executed as described in Algorithm 1.

1) *Security-Efficiency Trade-Off*: FL-MPS formulates parameter protection as a constrained optimization problem. For the j -th LoRA parameter, the negotiation output is denoted by

$$T_j = (s_j, r_j, p_j), \quad (1)$$

where $s_j \in \{0, 1\}$ indicates whether anchor negotiation succeeds, r_j is the representative client, and p_j is the residual precision. The objective is

$$\begin{aligned} \min_{\{s_j, p_j\}_{j=1}^M} & \sum_{j=1}^M [C_j(s_j, p_j) + \lambda Q_j(p_j) + \beta E_j(s_j, p_j)] \\ \text{s.t.} & \quad p_{\min} \leq p_j \leq p_{\max}, \\ & \quad s_j \in \{0, 1\}, \quad j = 1, \dots, M. \end{aligned} \quad (2)$$

Here, $C_j(s_j, p_j)$ denotes the computation and communication cost, $E_j(s_j, p_j)$ denotes the plaintext information exposure, and $Q_j(p_j)$ denotes the approximation error introduced by residual truncation. Specifically, $E_j(s_j, p_j)$ captures the leakage caused by transmitting residual bits in plaintext. When $s_j = 0$, the whole parameter is encrypted by RBPHE and $E_j(s_j, p_j)$ is minimized. When $s_j = 1$, the high-order anchor is encrypted while the residual is uploaded in plaintext, so $E_j(s_j, p_j)$ increases with the negotiated residual precision p_j . The coefficients λ and β control the relative importance of model utility and privacy leakage, respectively.

This formulation captures the core design goal of FL-MPS: minimizing cryptographic and communication overhead while penalizing both information leakage and aggregation error. A smaller p_j reduces communication cost and leakage but increases approximation error, whereas a larger p_j improves aggregation fidelity at the cost of exposing more residual bits. If anchor negotiation fails, FL-MPS sets $s_j = 0$ and falls back to full encryption, which minimizes plaintext leakage but increases cryptographic cost. Since solving this mixed-integer problem exactly at every round is expensive, FL-MPS uses anchor consistency and importance-aware precision negotiation as an efficient online approximation. In our experiments, the privacy-efficiency behavior of this objective is reflected through the negotiated precision range $p \in [p_{\min}, p_{\max}]$ and the negotiation threshold m . The threshold m does not appear as an independent optimization variable, but it determines the negotiation outcome s_j : a larger m imposes a stricter agreement condition and causes more parameters to fall back to full encryption, while a smaller m increases the proportion of split-protected parameters.

2) *Anchor Negotiation*: Each client P_i maintains a set of LoRA update components and flattens them into a one-dimensional vector

$$\text{vec}(\Delta\theta^i) = (\theta_1^i, \theta_2^i, \dots, \theta_M^i),$$

where each element θ_j^i represents a LoRA update encoded in a fixed-length binary format (e.g., IEEE-754 32-bit float or 64-bit double). During the t -th communication round, the j -th LoRA

update element of client P_i is denoted by

$$\theta_{j,t}^i, \quad i \in [1, N], \quad j \in [1, M].$$

In traditional FL, client P_i directly uploads $\theta_{j,t}^i$ to the parameter server (PS). In secure model aggregation (SMA), P_i instead uploads $\text{Enc}(\theta_{j,t}^i)$, which encrypts every parameter uniformly. However, fully encrypting all LoRA parameters in every round results in substantial computational and communication overhead, especially when LoRA ranks, or client numbers, are large.

To address this issue, our framework introduces a lightweight preprocessing step before uploading. Specifically, for each update element $\theta_{j,t}^i$, client P_i selects an *anchor* $a_{j,t}^i$. Since each $\theta_{j,t}^i$ is treated as a fixed-length binary string, we extract anchors from the significant bits—the sign bit and exponent bits in the IEEE-754 representation.

Formally, let $\theta_{j,t}^i$ consist of S bits. We define the anchor extracted by client P_i as

$$a_{j,t}^i = \text{ExtractBits}(\theta_{j,t}^i, 0, S),$$

where S bits correspond to the sign and exponent. These bits capture the coarse-grained scale and direction of the update and tend to be more stable across clients, which makes them suitable for cross-client negotiation.

For security reasons, participants cannot make proposals in plaintext. Assume that a secure hash function $H(\cdot)$ is available globally and a key pair (pk_i, sk_i) is set up beforehand for each P_i for secure communication. Then the negotiation begins with a global random number generation. Each P_i generates a random number s^i and encrypts it as $\tilde{s}_{pk_j}^i = \text{Enc}(pk_j, s^i)$. Then P_i sends the message to P_j , $i, j \in [1, n]$, $i \neq j$. Upon receiving $\tilde{s}_{pk_i}^j$ from other participants, P_i decrypts the message and obtains $s^j = \text{Dec}(sk_i, \tilde{s}_{pk_i}^j)$. In this way, each participant P_i can calculate a global random number $\bar{s} = \sum_{j=1}^n s^j$. In every iteration, P_i calculates $h_{j,t}^i = H(a_{j,t}^i \oplus \bar{s})$ and sends $h_{j,t}^i$ to the PS as an anchor proposal of a parameter $\theta_{j,t}^i$, where $\oplus$ defines an XOR operation.

After clients upload their local updates, PS identifies representative participants for each model parameter. For a parameter θ_j , the PS sorts the proposed anchor values from all clients and counts the frequency of each unique value. If a value appears at least m times, the corresponding client is randomly selected as the representative for that parameter; otherwise, a placeholder is used to indicate no valid representative.

3) *Precision Indicator Negotiation*: To enable computation- and communication-efficient parameter exchange in our federated LoRA training framework, we introduce an importance-driven precision indicator negotiation mechanism. This module determines the bit-level precision used for each LoRA residual update by leveraging the heterogeneous parameter importance across clients. The negotiation protocol ensures that highly influential parameters are transmitted with higher fidelity, while less critical parameters adopt lower-precision encodings to reduce overall overhead.

Following the idea of AdaLoRA, each client computes an importance score for every updated LoRA triplet after local fine-tuning. Let the LoRA update be $\Delta W_i = B_i A_i$ for client i . For the j -th component, we define its importance as:

$$s_{i,j} = \bar{I}_{i,j} \cdot \bar{U}_{i,j}, \quad (3)$$

where $\bar{I}_{i,j}$ denotes the exponential moving average (EMA) of the gradient-based sensitivity $I_{i,j} = |w_{i,j} \cdot g_{i,j}|$, with $w_{i,j}$ being the parameter value and $g_{i,j}$ its corresponding gradient. $\bar{U}_{i,j}$ quantifies the local uncertainty of the sensitivity estimate, computed as the EMA of the deviation between the current sensitivity $I_{i,j}$ and its smoothed value $\bar{I}_{i,j}$. This combined score captures both the parameter's impact on the loss and the stability of the estimate, providing a principled basis for dynamic triplet selection or precision allocation.

Each client maps its continuous importance scores into an integer-valued precision indicator, $p_j^i \in \mathbb{Z}^+$, which determines the bit-width used for the corresponding residual. Higher importance values are mapped to higher precision levels so that critical updates can be preserved under quantization and encryption.

To reach a global consensus, PS collects the precision proposals $\{p_j^i\}$ from all clients and performs a mode operation to derive the final negotiated precision:

$$p_j = \text{negotiate}(\{p_j^1, p_j^2, \dots, p_j^M\}).$$

This value is treated as the globally accepted bit-width for the j -th parameter's residual. The negotiation process occurs jointly with the anchor negotiation module, although the precision indicators are exchanged in plaintext.

The PS communicates the negotiation outcome using a structured triplet for each parameter:

$$T_j = (s_j, r_j, p_j), \quad (4)$$

where $s_j \in \{0, 1\}$ indicates whether consensus was reached, r_j denotes the representative client index used for anchor extraction, and p_j specifies the final precision level.

D. Aggregation Phase

The aggregation phase in our FL-MPS framework is designed to securely and efficiently combine model parameters from multiple clients, as shown in Algorithm 2. It occurs after the negotiation phase, where clients share information about parameter precision and agree on the best representations for each parameter. This phase ensures the preservation of privacy while reducing communication overhead through the use of model parameter splitting and homomorphic encryption.

In this phase, after completing the negotiation, each client receives a triplet table $T = (s, r, p)$. If the negotiation for a parameter fails (i.e., $s_j = 0$), the client encrypts the entire parameter using RBPHE to ensure full confidentiality during aggregation. When the negotiation succeeds (i.e., $s_j = 1$), the uploading behavior becomes parameter-dependent: only the representative client, identified by the condition r_j being equal to its own client ID, encrypts and uploads the anchor, corresponding to the first S bits of the parameter. All clients, regardless of

whether they are selected as the representative, upload the residual part ($S, S + p_j$) in plaintext. This split transmission strategy substantially reduces communication overhead while preserving the privacy of the most sensitive portion of the parameter.

After receiving all encrypted anchors and plaintext residuals, the parameter server (PS) performs aggregation strictly following the triplet table $T = (s, r, p)$ associated with each parameter. For parameters whose negotiation fails, the PS directly aggregates the fully encrypted values by operating homomorphically on the ciphertexts; specifically, RBPHE allows secure summation in the encrypted domain without exposing any intermediate information. In contrast, for parameters with successful negotiation, the PS processes the anchor and residual parts separately. The anchored segment, contributed only by the representative client, is forwarded unchanged to all clients in the next round. The residual segment, collected in plaintext from all clients, is averaged at the PS and the aggregated residual is then distributed back to the clients. Through this split aggregation strategy, FL-MPS preserves the confidentiality of high-sensitivity parameter bits while significantly reducing communication and computation overhead on the server side.

Overall, the FL-MPS protocol integrates parameter splitting, precision-aware negotiation, and hybrid encrypted-plaintext aggregation into a unified workflow. By isolating the high-sensitivity anchor bits and protecting them with homomorphic encryption while allowing low-sensitivity residuals to be transmitted and aggregated in plaintext, the proposed method achieves a fine-grained balance between privacy preservation and communication efficiency. The negotiation mechanism further reduces unnecessary encryption overhead by enabling clients to collaboratively agree on representative anchors and shared precision levels. Together, these design choices make FL-MPS particularly well-suited for large-scale federated LoRA fine-tuning, where the volume and sparsity of parameter updates naturally align with the split-and-protect strategy embedded into our framework.

IV. SECURITY ANALYSIS

A. Security of RBPHE

Intuitively, RBPHE guarantees the irrecoverability of parameters for SMA since μ_1 is encrypted under PHE and μ_2 is masked with a random value. However, one may still be concerned about the semantic information leaked by RBPHE, e.g., whether μ_2 promotes the advantage of an adversary $\mathcal{A}$ to disclose private information. Therefore, we formally prove the indistinguishability under the chosen plaintext attack (IND-CPA) of RBPHE. The security game of IND-CPA of RBPHE can be briefly abstracted by the following steps:

- 1) $\mathcal{A}$ chooses θ^0, θ^1 for a participant with pk .
- 2) The participant randomly picks $b \xleftarrow{\$} \{0, 1\}$ and sends $c \leftarrow \text{Encrypt}(R, \{\mathcal{P}, \mathcal{Q}\}, pk, \theta^b)$ to $\mathcal{A}$.
- 3) As long as $\mathcal{A}$ desires, it can further request the ciphertext of any θ from the participant.
- 4) $\mathcal{A}$ outputs b' and wins if $b' = b$.

Algorithm 2: Parameter-Level Aggregation of FL-MPS.**INPUT:** Number of clients N , anchor length S , Negotiation table $T = \{(s_j, r_j, p_j)\}_{j=1}^M$ **OUTPUT:** Updated global model parameters**ClientUpload i**

```

1: for  $j \leftarrow 1$  TO  $M$  do
2:   if  $s_j = 0$  then
3:      $c_j^i \leftarrow \text{RBPHE.Enc}(\theta_j^i)$ 
4:     Upload  $c_j^i$ 
5:   else
6:     if  $r_j = i$  then
7:        $ca_j^i \leftarrow \text{RBPHE.Enc}(a_j^i)$ 
8:       Upload  $ca_j^i$ 
9:     end if
10:     $u_j^i \leftarrow \text{Extract residual bits in } (S, S + p_j)$ 
11:    Upload  $d_j^i$ 
12:   end if
13: end for

```

ServerAggregation

```

14: for  $j \leftarrow 1$  TO  $M$  do
15:   if  $s_j = 0$  then
16:      $c_j \leftarrow \sum_{i=1}^N c_j^i$ 
17:   else
18:      $ca_j \leftarrow \text{Anchor from } r_j \text{ client } ca_j^i$ 
19:      $u_j \leftarrow \frac{1}{N} \sum_{i=1}^N u_j^i$ 
20:   end if
21: end for
22: Distribute global model

```

We first focus on μ_2 generated by the *Encrypt* algorithm because it is exposed to the adversary $\mathcal{A}$ directly. $\mathcal{A}$ can decompose μ_2 to the residues $\{\langle \theta_i \rangle_{q_i} | 1 \leq i \leq N\}$ (and $\langle \theta_0 \rangle_{q_0} = 1$) with the modulus $\{q_i | 1 \leq i \leq N\}$. Since each $\langle \theta_i \rangle_{q_i}$ is masked by $r \cdot \frac{q_i-1}{2}$ within $\mathbb{Z}_{q_i}$, we claim that there only exist two strategies for $\mathcal{A}$ to win the security game with a non-negligible advantage. Given $\theta^0, \theta^1, \langle \theta \rangle = \{\langle \theta_i \rangle_{q_i} | 1 \leq i \leq N\}$, $\mathcal{A}$

- computes the difference between each two residues $\langle \theta_i \rangle_{q_i} - \langle \theta_j \rangle_{q_j}$ for any $i \neq j$;
- or solves r with θ^0 and $\langle \theta \rangle$, or θ^1 and $\langle \theta \rangle$.

Theorem IV.1: Given $\theta = \{\theta_i | 1 \leq i \leq \mathcal{B}\}$, the advantage for an adversary $\mathcal{A}$ to distinguish $\langle \theta_i \rangle_{q_i} - \langle \theta_j \rangle_{q_j}$ ($i \neq j$) from the difference $v_i - v_j$ of two random values $v_i \in \mathbb{Z}_{q_i}$ and $v_j \in \mathbb{Z}_{q_j}$ is negligible.

Proof: For $k \in \{i, j\}$, $\langle \theta_k \rangle_{q_k}$ has the following form:

$$\langle \theta_k \rangle_{q_k} = \tilde{\theta}_k + r(q_k - 1)/2 \pmod{q_k},$$

where $\tilde{\theta}_k = \lceil \frac{\theta_k + R}{2R} \cdot (q_k - 1) \rceil$. Since q_k is a prime, $\frac{q_k-1}{2}$ is a generator of $\mathbb{Z}_{q_k}$. With the knowledge of θ_i and θ_j , the consistency of $\langle \theta_i \rangle_{q_i} - \langle \theta_j \rangle_{q_j}$ can be reduced to the indistinguishability between $\pi_1 = r \cdot \frac{q_i-1}{2} \pmod{q_i} - r \cdot \frac{q_j-1}{2} \pmod{q_j}$ and $\pi_2 = v_i - v_j$ of two random values v_i, v_j . Generally, r can be redefined by q_i and q_j :

$$r = a_i \cdot q_i + b_i = a_j \cdot q_j + b_j,$$

where $a_i, a_j, b_i, b_j \in \mathbb{Z}$. Therefore, π_1 is statistically identical to $b_i \cdot \frac{q_i-1}{2} \pmod{q_i} - b_j \cdot \frac{q_j-1}{2} \pmod{q_j}$. Since r is randomly picked and $q_i \neq q_j$, b_i and b_j are independently random to $\mathcal{A}$. In other words, the adversary $\mathcal{A}$ can hardly distinguish π_1 from π_2 . $\square$

Theorem IV.2: Given $\theta = \{\theta_i | 1 \leq i \leq \mathcal{B}\}$, the advantage for an adversary $\mathcal{A}$ to solve r from $\langle \theta \rangle = \{\langle \theta_i \rangle_{q_i} | 1 \leq i \leq \mathcal{B}\}$ is negligible under the hardness of Hilbert's tenth problem [34].

Proof: To secure the consistency of r , we prove that $\mathcal{A}$ cannot determine whether there exists a solution of r , or recover r with the following equations in polynomial time:

$$\left\{ \langle \theta_i \rangle_{q_i} = \tilde{\theta}_i + r \cdot \frac{q_i - 1}{2} \pmod{q_i} \mid 1 \leq i \leq \mathcal{B} \right\},$$

where $\tilde{\theta}_i = \lceil \frac{\theta_i + R}{2R} \cdot (q_i - 1) \rceil$. It is equivalent to solving r and n_i from the following equation under the constraint that $n_i \in \mathbb{Z}$.

$$\begin{pmatrix} \frac{q_1-1}{2} & q_1 & 0 & \cdots & 0 \\ \frac{q_2-1}{2} & 0 & q_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{q_{\mathcal{B}}-1}{2} & 0 & 0 & \cdots & q_{\mathcal{B}} \end{pmatrix} \begin{pmatrix} r \\ n_1 \\ \vdots \\ n_{\mathcal{B}} \end{pmatrix} = \begin{pmatrix} \tilde{\theta}_1' \\ \tilde{\theta}_2' \\ \vdots \\ \tilde{\theta}_{\mathcal{B}}' \end{pmatrix},$$

where $\tilde{\theta}_i' = \langle \theta_i \rangle_{q_i} - \tilde{\theta}_i$. Therefore, the advantage of solving r is reduced to solving $\mathcal{B} + 1$ integers $(r, n_1, \dots, n_{\mathcal{B}})$ with the above $\mathcal{B} + 1$ Diophantine equations, which is a case of Hilbert's tenth problem under the constraints that $0 \leq r \leq 2^\lambda$ and $0 \leq n_i \leq \frac{2^\lambda}{q_i}$. Solving the equation has been proved to be NP-complete [35] and it has been proved that the Hilbert tenth problem is undecidable for polynomials with 13 variables [36]. Therefore, we can set a batch size no less than the lower bound 13 to ensure that $\mathcal{A}$ cannot solve r . $\square$

Theorem IV.3: Assuming Theorems IV.1 and IV.2 hold, RBPHE achieves IND-CPA if PHE achieves IND-CPA.

Proof: Recall that $(\mathcal{P}, \mathcal{Q}, pk)$ are public parameters generated by the *Setup* algorithm. We construct a probabilistic polynomial-time (PPT) simulator $\mathcal{S}$ as follows. Taking as input $(\mathcal{P}, \mathcal{Q}, pk)$ and a vector θ , $\mathcal{S}$ outputs $\perp$ if $|\theta'| > |\mathcal{P}|$. Otherwise, $\mathcal{S}$ picks $v \xleftarrow{\$} \mathbb{Z}_n^{|\theta|}$ and $\mu' \xleftarrow{\$} \mathbb{Z}_n$ uniformly at random, where $\mathbb{Z}_n$ is the input space of PHE using the same security parameter as RBPHE. Then $\mathcal{S}$ evaluates $c' \leftarrow P\text{Encrypt}(pk, \mu')$ and encodes v with $\mathcal{Q}$ to obtain μ_2' . Finally, $\mathcal{S}$ outputs (c', μ_2') . For any encoding range R , we prove the indistinguishability $\text{Encrypt}(R, \{\mathcal{P}, \mathcal{Q}\}, pk, \theta) \stackrel{c}{\approx} \mathcal{S}(\{\mathcal{P}, \mathcal{Q}\}, pk, \theta)$ via the following hybrid argument:

Hyb₀ Taking as input $(R, \{\mathcal{P}, \mathcal{Q}\}, pk, \theta)$, the algorithm *Encrypt* of RBPHE outputs (c, μ_2) .

Hyb₁ Same as **Hyb₀** except that the algorithm picks $\mu' \xleftarrow{\$} \mathbb{Z}_n$ instead of encoding θ to μ_1 with $\mathcal{P}$ and encrypting μ_1 to c . The algorithm evaluates $c' \leftarrow P\text{Encrypt}(pk, \mu')$ and outputs (c', μ_2) . Since PHE achieves IND-CPA, this hybrid is indistinguishable from **Hyb₀**.

Hyb₂ Same as **Hyb₁** except that the algorithm picks $v \xleftarrow{\$} \mathbb{Z}_n^{|\theta|}$ and encodes v to μ_2' with $\mathcal{Q}$ instead of encoding θ to μ_2 with $\mathcal{Q}$. Assuming Theorem IV.2 holds, a PPT adversary can distinguish μ' from μ with a negligible probability. Therefore, this

hybrid outputs the view of $\mathcal{S}(\{\mathcal{P}, \mathcal{Q}\}, pk, \theta)$ and is statically identical to H_{yb_1} .

□

Given the proof of security of RBPHE, we can directly conclude the security of the RBPHE-based aggregation protocol. For parameters whose negotiation fails, FL-MPS falls back to full encrypted aggregation using RBPHE. For parameters whose negotiation succeeds, the high-order anchor bits are encrypted by the representative client using RBPHE, while only the negotiated residual bits are uploaded in plaintext. Therefore, the encrypted parts inherit the semantic security of RBPHE, and the remaining FL-MPS-specific questions are whether hashed anchor negotiation and plaintext residual transmission reveal enough information for reconstruction.

B. Residual Leakage and Inversion Resistance

During anchor negotiation, clients do not send anchor values directly. Instead, each client sends a hash value computed from the anchor and a round-dependent random seed. Under the preimage resistance of the hash function, the server can identify whether multiple clients submit the same protected anchor in the same round, but cannot recover the anchor value itself. Therefore, anchor negotiation reveals an equality pattern used for representative selection, not the sign, exponent, or magnitude of any individual parameter.

Although FL-MPS uploads residual bits in plaintext for successfully negotiated parameters, the residual alone does not reveal the complete LoRA update. For a parameter $\theta_{j,t}^i$, FL-MPS decomposes its representation into an anchor $a_{j,t}^i$, a residual $u_{j,t}^i$ with precision p_j , and a truncated tail $z_{j,t}^i$. The anchor contains high-order numerical information, such as sign, exponent, and coarse magnitude, while the residual only represents a limited-precision offset within the range determined by the hidden anchor. Given only $u_{j,t}^i$, the server still needs to infer both $a_{j,t}^i$ and $z_{j,t}^i$. If the anchor length is S and the truncated tail length is d_r , then each observed residual is consistent with 2^{S+d_r} possible parameter values.

Theorem IV.4: Given a plaintext residual $u_{j,t}^i$ with precision p_j , and assuming that the anchor $a_{j,t}^i$ is hidden from the semi-honest server, the server's remaining uncertainty about $\theta_{j,t}^i$ is at least $S + d_r$ bits.

Proof: The residual fixes only the middle p_j bits of the parameter representation. The high-order anchor bits and the truncated lower-order bits remain unknown. Thus, for each residual pattern, there exist 2^{S+d_r} feasible parameter values consistent with the server's observation. Since gradient inversion attacks require sufficiently accurate update values and their cross-coordinate structure, the plaintext residual alone is insufficient to reconstruct the original LoRA update or the client's private data. □

We further consider multi-round and cross-parameter observations. Plaintext residuals can reveal bounded low-order numerical fragments over multiple rounds, and therefore they

should not be interpreted as zero leakage. However, FL-MPS does not expose the corresponding high-order anchors to the server, and parameters that fail to reach stable consensus fall back to full encryption. As a result, the server observes fragmented residual coordinates without the full scale and direction information needed to assemble an accurate client update vector. Cross-parameter correlations may provide auxiliary statistical information, but they do not remove the ambiguity of the hidden anchors. If an anchor were leaked outside the protocol, the residual would indeed become more informative; this is why FL-MPS encrypts anchors and treats anchor visibility as part of the protected state.

C. Adaptive and Colluding Participants

For client-side inference, an honest-but-curious client may subtract its own residual from the aggregated residual, but this only reveals the sum of residuals from the remaining clients. As long as fewer than $N - 2$ clients collude, at least two honest clients remain mixed in the aggregate, and no individual residual can be uniquely determined. Encrypted anchors and fully encrypted parameters similarly do not reveal another client's individual contribution to non-representative clients.

We next consider a stronger collusion scenario beyond the semi-honest model, where malicious clients coordinate anchor proposals or randomness contributions to manipulate negotiation. The threshold m controls the minimum number of matching protected anchors required for split protection. A coalition controlling at least m clients may fabricate consensus and enlarge the plaintext residual surface, whereas a smaller coalition fails to meet the agreement condition and triggers full RBPHE fallback. Thus, m does not eliminate active collusion, but raises the coalition size required to manipulate negotiation.

The randomness used in anchor hashing has a similar boundary. Since the global seed is generated from client-side contributions, it remains unpredictable as long as at least one participant contributes fresh randomness honestly. Under this condition, the same anchor is mapped to different hash values across rounds, preventing direct cross-round linking of anchor hashes. Moreover, assuming the preimage resistance of the hash function [37], anchor values themselves are still not directly recovered from the hashes.

Finally, representative selection follows the negotiation outcome and is performed for each parameter in each round, rather than assigning a fixed representative role to any client. Stable anchors may still cause some clients to be repeatedly selected across epochs, leading to workload imbalance or persistent equality patterns. Since representatives upload encrypted anchors rather than plaintext anchors, repeated selection does not directly expose anchor values, but it may increase the risk of persistent leader control or collusive manipulation. This can be mitigated in deployment through representative rotation or verifiable election mechanisms, as suggested by BFT and PoS-style consensus studies [38], [39]. Designing a full consensus-based election layer is orthogonal to FL-MPS and is left as future work.

TABLE I
COMPARISON OF FL-MPS WITH BASELINES

	FedIT	FLoRA	FedShield-LLM	DP-LoRA	FL-MPS
Dolly	39.28	40.66	39.91	35.13	40.31
Wizard	40.28	42.45	39.37	39.57	41.92

V. EVALUATION

A. Experimental Setup

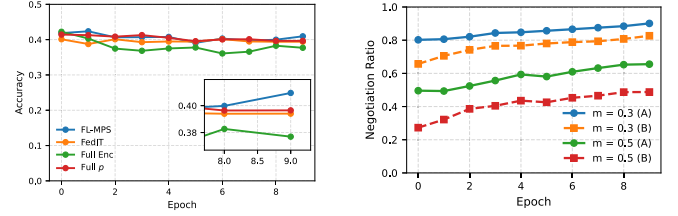
We have implemented FL-MPS and evaluated our solution comprehensively. All experiments were conducted on a Linux server with Intel(R) Xeon(R) Gold 6248 CPU running at 2.50 GHz on 20 cores and 232 GB of RAM. In our FL setup, we assume the presence of 10 clients. We proceed to apply the Shepherd framework’s second data partitioning technique to divide the residual data from the Dolly dataset into 10 distinct portions. Each of these portions corresponds to an individual client’s local instruction dataset. We conduct a total of 10 communication rounds. Each client performs one epoch of local training with their respective instruction datasets. We initialize the model with the 7B LLaMA model. The model remains frozen during training. We apply LoRA to all linear layers with a rank of 4 to boost adaptation capabilities. We employ the Adam optimizer to update LoRA parameters with a batch size of 64 and a learning rate of $3e-4$. We set the maximum input sequence length to 512. The derived model is referred to as Shepherd-7B.

Besides, we investigate how key system parameters affect the performance of FL-MPS, with a particular focus on m and p . These two parameters play a critical role in the negotiation phase. Specifically, m denotes the minimum ratio required for parameter agreement, while p controls the numerical precision used during parameter protection and aggregation. Unless otherwise stated, we set $m = 0.3$ and vary p within the range $[5, 10]$ as the default configuration.

B. Experimental Result

Performance: We compare FL-MPS with FedIT and FLoRA, two representative federated LoRA fine-tuning baselines [12], [13], as well as two privacy-preserving baselines, FedShield-LLM [16] and DP-LoRA [15]. FedIT integrates LoRA into the FedAvg framework, while FLoRA improves aggregation by introducing a stacking strategy to mitigate averaging-induced errors. FedShield-LLM protects LoRA fine-tuning with homomorphic encryption and pruning, and DP-LoRA provides privacy protection by injecting Gaussian noise into LoRA adapter weights. For a fair comparison, all methods use an identical LoRA rank of 4. Table I summarizes the final performance on the MMLU benchmark. We evaluate model performance using accuracy on a multiple-choice question answering (Q&A) validation set. For each question, the model is required to select one option from a fixed set of candidate answers. A prediction is considered correct if the selected option matches the ground-truth answer.

FL-MPS achieves competitive performance compared with existing baselines on both the Dolly and Wizard subsets. As



(a) Model accuracy.

(b) Negotiation ratio.

Fig. 4. Training performance and negotiation dynamics of FL-MPS.

TABLE II
ABLATION STUDY OF FL-MPS VARIANTS

Variant	Protection strategy	Dolly	Wizard
Full Enc	Full-para RBPHE aggregation	39.47	40.49
Full p	Split-agg with full residual precision	39.64	40.53
FL-MPS	Adaptive split with negotiation	40.31	41.92

shown in Table I and Fig. 4(a), FL-MPS improves the accuracy from 39.28 to 40.31 on Dolly and from 40.28 to 41.92 on Wizard when compared with FedIT. Compared with the privacy-preserving baselines, FL-MPS also achieves higher accuracy than FedShield-LLM and DP-LoRA on both subsets. This suggests that uniformly applying strong protection or noise injection may affect model utility, whereas FL-MPS protects high-order anchor information with RBPHE while exposing only bounded residuals with negotiated precision. Although FLoRA achieves the highest accuracy, FL-MPS remains competitive while providing parameter-level privacy protection through negotiation and split aggregation.

We further conduct an ablation study in Table II. Full Enc applies full-parameter RBPHE aggregation, while Full p uses full residual precision ($p = 23$) under the split aggregation framework. FL-MPS outperforms both variants, indicating that directly encrypting all parameters or aggregating full-precision residuals does not necessarily improve performance. Instead, the proposed negotiation and selective protection mechanisms introduce an implicit regularization effect by partially suppressing noisy or low-consensus LoRA updates during aggregation, thereby mitigating the accumulation of aggregation noise commonly observed in FedAvg-based LoRA fine-tuning. Moreover, the training curves reveal that Full Enc exhibits noticeably larger fluctuations during training. This instability can be attributed to the additional numerical error introduced during encrypted aggregation, where model parameters must be encoded and quantized before applying RBPHE. In contrast, FL-MPS selectively aggregates high-consensus parameters through negotiation, which mitigates the impact of these errors and results in a smoother optimization trajectory and more stable convergence.

Beyond final accuracy, we further examine the effectiveness of the negotiation mechanism by analyzing the evolution of successfully negotiated parameters across training epochs, as shown in Fig. 4(b). We observe that the proportion of parameters reaching consensus through negotiation increases steadily as training proceeds, indicating that clients gradually converge on a more consistent set of LoRA updates. This trend suggests that the

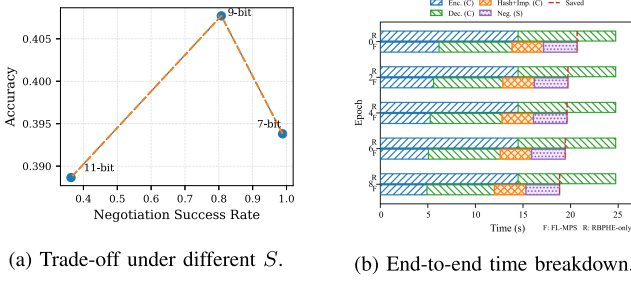


Fig. 5. Trade-off analysis and runtime breakdown of FL-MPS.

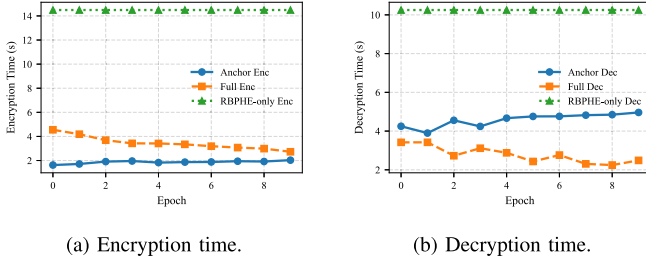


Fig. 6. Cryptographic time cost per epoch under different parameter protection strategies.

negotiation process is not only stable but also progressively effective in filtering out low-agreement updates, thereby enabling a larger fraction of parameters to be safely and jointly aggregated.

Fig. 5(a) illustrates the relationship between the negotiation prefix length S , negotiation success rate, and model accuracy. As the S increases, the negotiation success rate gradually decreases because longer S impose stricter matching conditions on parameter anchors, making it harder for clients to reach agreement during negotiation. In terms of model performance, the accuracy peaks at 9 bits. When the prefix is too short (e.g., 7 bits), the anchor representation becomes overly coarse, which may introduce minor inconsistencies in parameter protection decisions. In contrast, when the prefix becomes too long (e.g., 11 bits), the negotiation success rate drops noticeably, meaning that fewer parameters can be handled through the anchor-based mechanism and more parameters fall back to full protection, which introduces additional numerical errors during encrypted aggregation and slightly perturbs the training dynamics. As a result, 9 bits achieves the best trade-off between negotiation efficiency and model performance in our framework.

Overall, these results indicate that the proposed negotiation and protection mechanisms do not harm the learning effectiveness, and may even lead to marginal performance gains under certain settings.

Efficiency: We further evaluate the efficiency of our method by comparing the cryptographic overhead with a RBPHE-only baseline. Fig. 6(a) illustrates the encryption time per epoch, while Fig. 6(b) reports the corresponding decryption time. In both figures, we compare FL-MPS with the baseline that encrypts all parameters using RBPHE.

As the number of epochs increases, the negotiation success rates for both LoRA A and LoRA B gradually rise, indicating that parameter importance becomes more stable and consistent

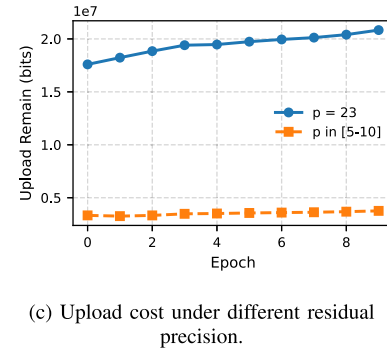
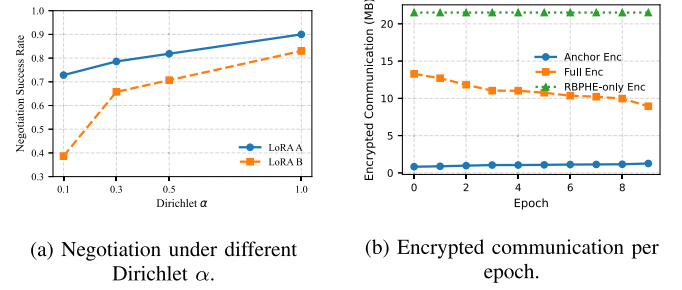


Fig. 7. Impact of client heterogeneity, encrypted communication, and residual precision on FL-MPS.

across clients. Correspondingly, the encryption and decryption time of the anchor part increases, since more parameters are identified as negotiable and thus protected.

At the same time, the encryption and decryption costs of the full part decrease, as fewer parameters remain fully protected. As a result, although the computational overhead shifts from the full part to the anchor part, the overall cryptographic cost remains significantly lower than that of the RBPHE-only encryption baseline throughout all epochs.

To further provide an end-to-end view of computational overhead, Fig. 5(b) reports the time breakdown of FL-MPS and the RBPHE-only baseline. Although FL-MPS introduces additional client-side hash and importance computation, and server-side negotiation costs, these costs are lightweight compared with the encryption and decryption overhead saved by avoiding full-parameter RBPHE aggregation. As a result, FL-MPS achieves lower total runtime while preserving encrypted protection for security-critical parameter segments.

Beyond computation time, FL-MPS also achieves notable communication reduction. As shown in Fig. 7(b), the encrypted payload associated with fully protected parameters decreases over training, while the communication cost of the anchor part slightly increases. Importantly, both components remain far smaller than the communication cost of encrypting all parameters with RBPHE. This asymmetric trend demonstrates that FL-MPS effectively replaces large-scale full encryption with lightweight anchor-based protection, resulting in consistently lower encrypted communication overhead.

Overall, this dynamic behavior highlights a key advantage of FL-MPS: rather than incurring uniformly high cryptographic and communication costs, the system adaptively reallocates protection effort toward a compact set of sensitive parameters,

TABLE III
IMPACT OF NEGOTIATION THRESHOLD m ON SYSTEM PERFORMANCE

	A	B	Enc-A	Enc-F	Dec-F	Dec-A	Acc
$m = 0.3$	0.90	0.83	2.04	2.74	2.48	4.96	0.405
$m = 0.5$	0.66	0.49	1.62	6.59	5.10	3.31	0.409

Notes: A/B denote the negotiation success rate of LoRA matrices A and B , respectively. Enc-A and Enc-F represent the average encryption time (in seconds) for anchor-only and full encryption. Dec-A and Dec-F denote the corresponding decryption time.

thereby achieving high efficiency while maintaining strong privacy guarantees.

The Impact of Key Parameters: We move forward to analyze the impact of key system parameters m and p on the performance of FL-MPS, in terms of negotiation success rate, cryptographic overhead, and model accuracy.

We study the impact of the negotiation threshold m by comparing two representative settings, $m = 0.3$ and $m = 0.5$. As reported in Table III, the negotiation success rates for both LoRA matrices A and B decrease as m increases. This is expected, since a larger m imposes a stricter requirement on cross-client parameter agreement, making successful negotiation more difficult.

As shown in Fig. 4(b), we further observe that the negotiation success rate of matrix A is consistently higher than that of matrix B under all configurations. This phenomenon aligns with prior observations in federated LoRA training, where matrix A exhibits stronger cross-client similarity, while matrix B captures more client-specific variations. As a result, parameters in A are more likely to satisfy the negotiation criterion.

Despite the reduced negotiation success rate under larger m , the overall model accuracy remains largely unchanged. Meanwhile, stricter negotiation leads to a higher proportion of parameters being protected by full encryption, resulting in increased cryptographic overhead. This highlights an inherent trade-off between negotiation flexibility and system efficiency. Based on this trade-off, we adopt $m = 0.3$ as the default setting to balance security, negotiation robustness, and runtime cost.

To further understand the effect of data heterogeneity, we vary the Dirichlet parameter α , where a smaller α indicates a more highly non-IID client distribution. As shown in Fig. 7(a), the negotiation success rate of LoRA A remains consistently high even under strong heterogeneity, suggesting that LoRA A preserves relatively stable cross-client anchor patterns. In contrast, LoRA B is more sensitive to client heterogeneity: when $\alpha = 0.1$, its negotiation success rate drops substantially, indicating that client-specific updates make anchor agreement more difficult. When negotiation fails more frequently, FL-MPS automatically falls back to full RBPHE encryption for those parameters. Therefore, extreme heterogeneity does not weaken privacy protection; instead, it reduces the fraction of split-protected parameters and increases cryptographic overhead. In the worst case, if most parameters fail to reach agreement, FL-MPS degenerates toward the full-encryption baseline, preserving confidentiality at the cost of lower efficiency gains.

We next investigate the influence of the precision parameter p . Specifically, we compare two settings: a moderate precision

range $p \in [5, 10]$ and a fixed high-precision baseline $p = 23$. As shown in Fig. 7(c), lowering p substantially reduces the remaining payload uploaded to the server, leading to improved communication efficiency. This result highlights that FL-MPS can safely operate under lower precision without sacrificing accuracy, while benefiting from reduced transmission overhead.

In summary, these results validate that FL-MPS is an effective and practical framework for secure and efficient federated fine-tuning of large language models, providing strong privacy protection with minimal overhead and no degradation in model utility.

VI. RELATED WORK

Recent studies have explored the integration of LoRA-based parameter-efficient fine-tuning into federated learning for large language models, aiming to reduce communication and computational overhead while preserving client-specific knowledge. FedIT [12] incorporates LoRA directly into the classical FedAvg framework [8] for instruction tuning, providing a straightforward method for collaborative model adaptation. Building on the observation that the A matrices primarily capture general knowledge and the B matrices encode client-specific features, FedSA-LoRA [40] uploads only the A matrices to the server, significantly lowering communication costs.

To address device heterogeneity, FLoRA [13] proposes a stacking-based aggregation scheme that allows clients to select LoRA ranks according to their local resource capacities. Similarly, FlexLoRA [41] dynamically adjusts local LoRA ranks and leverages singular value decomposition for weight redistribution, optimizing the use of heterogeneous device resources.

Beyond efficiency, privacy-preserving mechanisms have also been explored in LoRA-based FL. DP-LoRA [15] adds Gaussian noise to LoRA adapter weights, providing formal (ϵ, δ) -differential privacy guarantees while maintaining model accuracy. The compressed structure of LoRA reduces the required noise scale, making DP practical for large models. More recently, differentially private federated low-rank adaptation beyond fixed-matrix designs [42] further studies how to improve the privacy-utility trade-off when LoRA updates are no longer restricted to a fixed adapter matrix. In parallel, FedShield-LLM [16] introduces a federated fine-tuning framework that combines FL, LoRA, full homomorphic encryption (FHE), and pruning to enable scalable and secure training of LLMs on decentralized private datasets. SHE-LoRA [43] adopts selective homomorphic encryption for federated tuning with heterogeneous LoRA, reducing encryption overhead by protecting only selected LoRA components.

Our FL-MPS takes into account different learning phases of FL, aiming to only select the representative local model parameters as scheduled communication participants. The PS is only required to execute a simple alignment task for the selection instead of calculating the comparison of the whole model. The confidentiality of transmitted parameters can be enforced by HE protocols. Other security requirements, such as published model inference resistance [17] and poisoning attack defense [44], should be studied separately.

Our FL-MPS addresses both efficiency and privacy challenges in LoRA-enabled federated learning. Our framework leverages the observation that different LoRA parameters behave differently during training and therefore benefit from parameter-specific protection. We introduce a lightweight negotiation protocol that allows clients to collaboratively determine the protection mode for each parameter, combined with a residual encoding-based homomorphic encryption scheme that outperforms existing solutions in SIMD operations. This approach maintains model accuracy, reduces communication and computation overhead, and provides formal privacy guarantees, enabling scalable, secure, and resource-aware federated fine-tuning of LLMs.

VII. CONCLUSION

We propose a lightweight and privacy-preserving negotiation protocol that enables clients to collaboratively determine the protection mode of each LoRA parameter, combined with a residual encoding-based HE scheme that outperforms existing SIMD-based solutions. Besides, we are the first to leverage the varying behavior of LoRA parameters during training to apply differentiated protection, which effectively balances privacy, efficiency, and model accuracy. We note that FL-MPS is primarily designed for parameter-level privacy and efficiency, which means that we do not explicitly consider client heterogeneity in terms of data distribution or computational resources. Future work includes extending FL-MPS to handle heterogeneous clients more effectively and exploring adaptive strategies that dynamically adjust parameter protection and encryption based on resource constraints and data characteristics.

ACKNOWLEDGEMENT

The authors would like to thank the editor and anonymous reviewers for their valuable comments and constructive suggestions, which helped improve the quality and completeness of this work.

REFERENCES

- [1] OpenAI, "GPT-4 technical report," 2023. Accessed: Dec. 08, 2023. [Online]. Available: <https://openai.com/research/gpt-4>
- [2] Y. Lei, J. Yang, Y. Mu, H. Liang, and D. Jia, "Border of speech: A benchmark for understanding chinese offensive speech," *IEEE Trans. Comput. Social Syst.*, vol. 13, no. 3, pp. 3235–3247, Jun. 2026.
- [3] X. Wang et al., "Multimodal learning with next-token prediction for large multimodal models," *Nature*, vol. 650, pp. 327–333, 2026.
- [4] F. Rivas-Echeverria, L. T. Ramos, J. L. Ibarra, S. Zerpa-Bonillo, S. Arciniegas, and M. Asprino-Salas, "LegalBot-EC: An LLM-based chatbot for legal assistance in ecuadorian law," in *IEEE Access*, vol. 13, pp. 106817–106833, 2025.
- [5] E. Ullah, A. Parwani, M. M. Baig, and R. Singh, "Challenges and barriers of using large language models (LLM) such as ChatGPT for diagnostic medicine with a focus on digital pathology—a recent scoping review," *Diagn. Pathol.*, vol. 19, 2024, Art. no. 43.
- [6] California State Legislature, "California consumer privacy act of 2018," *California Civil Code*, 2018. [Online]. Available: <https://www.caprivacy.org/>
- [7] "Regulation (EU)/679 of the European parliament and of the council of 27," *Official J. Eur. Union*, vol. L 119, pp. 1–88, apr. 2016. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [8] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [9] R. Ye et al., "OpenFedLLM: Training large language models on decentralized private data via federated learning," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 6137–6147.
- [10] N. Carlini et al., "Extracting training data from large language models," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2633–2650.
- [11] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Representations*, 2022, Art. no. 3.
- [12] J. Zhang et al., "Towards building the federatedGPT: Federated instruction tuning," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2024, pp. 6915–6919.
- [13] Z. Wang et al., "Flora: Federated fine-tuning large language models with heterogeneous low-rank adaptations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 22513–22533.
- [14] Y. Gao, Y. Xie, H. Deng, and Z. Zhu, "Gradient inversion attack in federated learning: Exposing text data through discrete optimization," in *Proc. 31st Int. Conf. Comput. Linguist.*, 2025, pp. 2582–2591.
- [15] X.-Y. Liu et al., "Differentially private low-rank adaptation of large language model using federated learning," *ACM Trans. Manage. Inf. Syst.*, vol. 16, pp. 1–24, 2025.
- [16] M. J. Mia and M. H. Amini, "FedShield-LLM: A secure and scalable federated fine-tuned large language model," 2025, *arXiv:2506.05640*.
- [17] Y. Mao et al., "FLSwitch: Towards secure and fast model aggregation for federated deep learning with a learning state-aware switch," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, 2023, pp. 476–500.
- [18] H. Yangsibo et al., "Evaluating gradient inversion attacks and defenses in federated learning," in *Proc. 35th Conf. Neural Inf. Process. Syst.*, 2021, pp. 7232–7241.
- [19] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "BatchCrypt: Efficient homomorphic encryption for cross-silo federated learning," in *Proc. USENIX Annu. Tech. Conf.*, 2020, pp. 493–506.
- [20] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure single-server aggregation with (poly) logarithmic overhead," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 1253–1269.
- [21] Y. Aono et al., "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 5, pp. 1333–1345, May 2018.
- [22] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 911–926, 2020.
- [23] X. Guo et al., "VeriFL: Communication-efficient and fast verifiable aggregation for federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 1736–1751, 2021.
- [24] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 3–18.
- [25] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 739–753.
- [26] X. Luo, Y. Wu, X. Xiao, and B. C. Ooi, "Feature inference attack on model predictions in vertical federated learning," in *Proc. Int. Conf. Data Eng.*, 2021, pp. 181–192.
- [27] D. Pasquini, G. Ateniese, and M. Bernaschi, "Unleashing the tiger: Inference attacks on split learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 2113–2129.
- [28] B. Hitaj, G. Ateniese, and F. Perez-Cruz, "Deep models under the gan: Information leakage from collaborative deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 603–618.
- [29] L. Wang, J. Bian, L. Zhang, and J. Xu, "Adaptive LORA experts allocation and selection for federated fine-tuning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2025, pp. 76018–76045.
- [30] Q. Zhang et al., "Adaptive budget allocation for parameter-efficient fine-tuning," in *Proc. 11th Int. Conf. Learn. Representations*, 2023, pp. 1–17.
- [31] C. Baskin et al., "UNIQ: Uniform noise injection for non-uniform quantization of neural networks," *ACM Trans. Comput. Syst.*, vol. 37, pp. 1–15, 2021.
- [32] Z. Jiang, W. Wang, and Y. Liu, "FLASH: Additively symmetric homomorphic encryption for cross-silo federated learning," 2021, *arXiv:2109.00675*.
- [33] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2017, pp. 409–437.

- [34] D. Hilbert, "Mathematische probleme," in *Dritter Band: Anal. Grundlagen der Mathematik Physik Verschiedenes*. Berlin, Germany: Springer, 1935, pp. 290–329.
- [35] E. M. Gurari and O. H. Ibarra, "An NP-complete number-theoretic problem," *J. ACM*, vol. 26, no. 3, pp. 567–581, 1979.
- [36] Y. Matijasević and J. Robinson, "Reduction of an arbitrary diophantine equation to one in 13 unknowns," *Collected Works Julia Robinson*, vol. 6, 1996, Art. no. 235.
- [37] P. Rogaway and T. Shrimpton, "Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance," in *Fast Software Encryption*. Berlin, Germany: Springer, 2004, pp. 371–388.
- [38] G. Zhang, F. Pan, S. Tijanic, and H.-A. Jacobsen, "PrestigeBFT: Revolutionizing view changes in BFT consensus algorithms with reputation mechanisms," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 1930–1943.
- [39] Y. Vonlanthen, J. Sliwinski, M. Albarello, and R. Wattenhofer, "Banyan: Fast rotating leader BFT," in *Proc. 25th Int. Middleware Conf.*, 2024, pp. 494–507.
- [40] P. Guo, S. Zeng, Y. Wang, H. Fan, F. Wang, and L. Qu, "Selective aggregation for low-rank adaptation in federated learning," in *Proc. Int. Conf. Learn. Representations*, vol. 2025, 2025, pp. 99003–99027.
- [41] J. Bai, D. Chen, B. Qian, L. Yao, and Y. Li, "Federated fine-tuning of large language models under heterogeneous tasks and client resources," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 14457–14483.
- [42] M. Wen, J. Zhu, Y. Xu, Y. Zhou, and D. Han, "Differentially private federated low rank adaptation beyond fixed-matrix," in *Proc. Adv. Neural Inf. Process. Syst.*, 2026, pp. 114755–114786.
- [43] J. Liu, L. Yan, B. Li, L. Yu, and C. Shen, "SHE-LoRA: Selective homomorphic encryption for federated tuning with heterogeneous LoRA," in *Proc. Int. Conf. Learn. Representations*, vol. 2026, 2026, pp. 154701–154733.
- [44] Y. Mao, X. Yuan, X. Zhao, and S. Zhong, "Romoa: Robust model aggregation for the resistance of federated learning to model poisoning attacks," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2021, pp. 476–496.



Guanzhi Qiu is currently working toward the undergraduate degree in software engineering with Nanjing University, Nanjing, China. His research interests include federated learning security, privacy preservation for LLMs, and AI security.



Shijie Ding is currently working toward the undergraduate degree in software engineering from Nanjing University, Nanjing, China. His research interests include federated learning, physical AI, and large language model.



Wenjun Qian received the PhD degree from Peking University, Beijing, China, in 2024. She is currently an engineer with China Mobile Research Institute, engaging in frontier basic research on AI security. She has authored or coauthored multiple peer-reviewed papers in top international journals and conferences. Her research interests include differential privacy, federated learning, and security of LLMs.



Yuejuan Zhu received the BS degree in software engineering in 2025 from Nanjing University, Nanjing, China, where she is currently working toward the MS degree in software engineering. Her research interests include federated learning and privacy preservation for LLMs.



Yunlong Mao (Member, IEEE) received the BS and PhD degrees in computer science and technology from Nanjing University. He is currently an associate professor with the State Key Laboratory for Novel Software Technology, Nanjing University. His research interests include trustworthy artificial intelligence, machine learning security, and privacy preservation.



Sheng Zhong (Fellow, IEEE) received the BS and MS degrees in computer science from Nanjing University in 1996 and 1999, respectively, and the PhD degree in computer science from Yale University in 2004. He is currently a professor with the State Key Laboratory for Novel Software Technology, Nanjing University. His research interests include security, privacy, and economic incentives.